

Altimertik Interview Preparation - L2

10 August 2026 11:09

Question-1 (Medium)

You're building a customer support bot for a fintech company. A poorly written prompt is producing inconsistent, sometimes overly casual, and occasionally incorrect responses. The current prompt is just:

"You are a helpful assistant. Answer customer question."

How would you redesign this prompt, and what specific elements would you add? Write the improved prompt.

ML/AI concepts: prompt engineering

Element	Purpose
Role	Establishes the assistant's responsibility
Context	Defines the fintech / customer-support environment
Tone	Prevent overly casual or inconsistent responses
Scope	Defines what the bot should handle
Grounding	Reduces hallucination
Uncertainty handling	Prevents the model from guessing
Safety constraints	Important for financial applications
Response format	Makes answer consistent
Escalation rules	Handles cases the AI shouldn't resolve
Examples	Demonstrates the expected behavior
Priority rules	Helps resolve competing instructions

Question-2 (Hard)

You are given a trained PyTorch classification model that perform well offline.
You now need to deploy it as an inference service and ensure correctness, performance, and reproducibility

Python: Python ML Frameworks

Ans: How I would approach it

I would divide the deployment into 6 stages:

Trained PyTorch Model

↓



1. Model Validation

├— Offline metrics

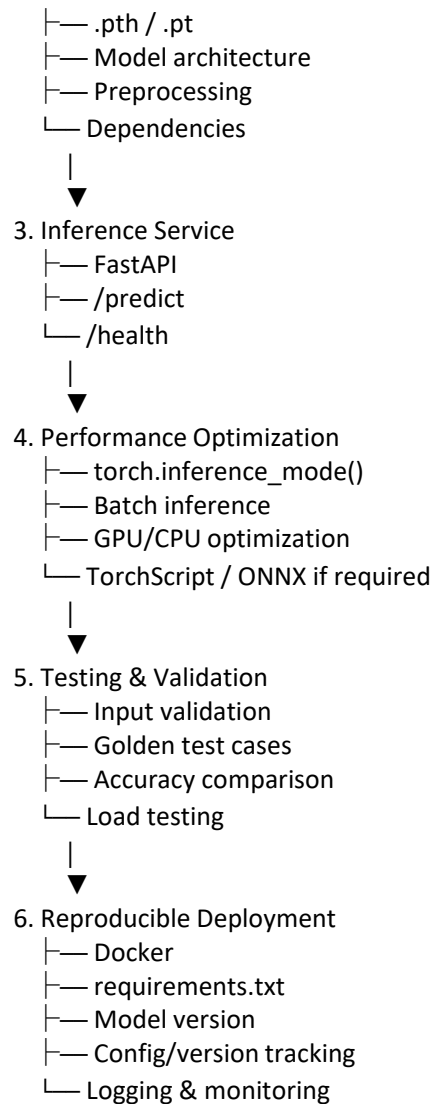
├— Test predictions

└— Edge cases

↓



2. Model Packaging



Correctness:

The first thing I would not do is immediately expose the model through an API.

I would first verify that the deployed model produces the same results

Important Checks:

Model loading

```
model = MyClassifier(...)
model.load_state_dict(torch.load("model.pth"))
model.eval()
```

Use:

```
model.eval()
```

Because layers such as **Dropout** and **BatchNorm** behave differently during **training and inference**.

For Inference:

```
With torch.inference_mode():
    output = model(input_tensor)
```

`Torch.inference_mode()` **prevent unnecessary autograd tracking** and is preferable for inference

workloads.

Preprocessing must be identical:

A common production bug is:

Training: Resize -> Normalize -> tensor

Production: Resize -> different normalize -> tensor

The model may technically be correct, but prediction become wrong.

So I would version and package:

- Image/text preprocessing
- Normalization values
- Tokenizer if applicable
- Input dimensions
- Class mapping
- Model weights
- Model architecture
- Inference configuration

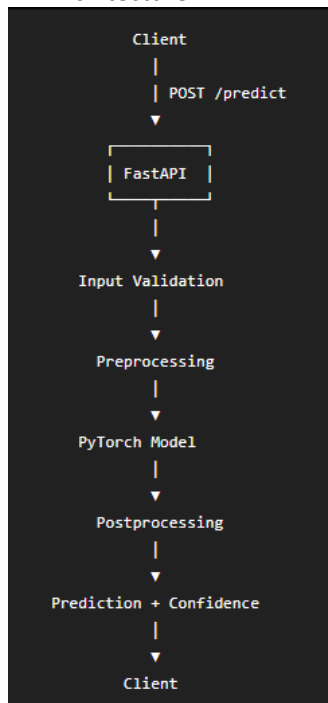
Ex: 0->cat, 1->dog, 2->horse

The class mapping must remain consistent between training and production.

Build an inference API:

Python-based deployment, I would typically use FastAPI

Ex: Architecture



Code: simplified implementation could look like

```
from fastapi import FastAPI
import torch
```

```
app = FastAPI()
```

```

model = MyClassifier()
model.load_state_dict(torch.load("model.pth", map_location="cpu"))
model.eval()

```

```

@app.get("/health")
def health():
    return {"status": "healthy"}

```

```

@app.post("/predict")
def predict(request: PredictionRequest):

    tensor = preprocess(request.input)

    with torch.inference_mode():
        output = model(tensor)

    prediction = torch.argmax(output, dim=1)

    return {
        "class_id": int(prediction.item())
    }

```

In production, I would also add proper exception handling, schema validation, structured logging, authentication/authorization where required, and request IDs.

Performance:

Once correctness is established, I would optimize performance.

First optimization

Use:

```

model.eval()

```

```

with torch.inference_mode():
    output = model(x)

```

Other techniques

Depending on the workload:

Technique	Benefit
Torch.inference_mode()	Removes inference-time autograd overhead
Batch inference	Better hardware utilization
GPU inference	Lower latency/higher throughput for suitable models
Mixed precision	Faster/lower memory usage on supported hardware
Model quantization	Lower memory and potentially faster CPU inference
TorchScript	Possible deployment optimization
ONNX	Portable optimized inference
TensorRT	High-performance NVIDIA inference
Connection pooling	Better service efficiency
Async/request batching	Better throughput for high traffic

The important interview point is:

I would optimize only after measuring the baseline.

Correctness testing:

I would create a golden dataset.

Ex:

Input A → Expected class = 2

Input B → Expected class = 0

Input C → Expected class = 1

Then compare:

Offline Model Prediction

VS

Production API Prediction

They should match within an acceptable tolerance.

For classification, I would validate

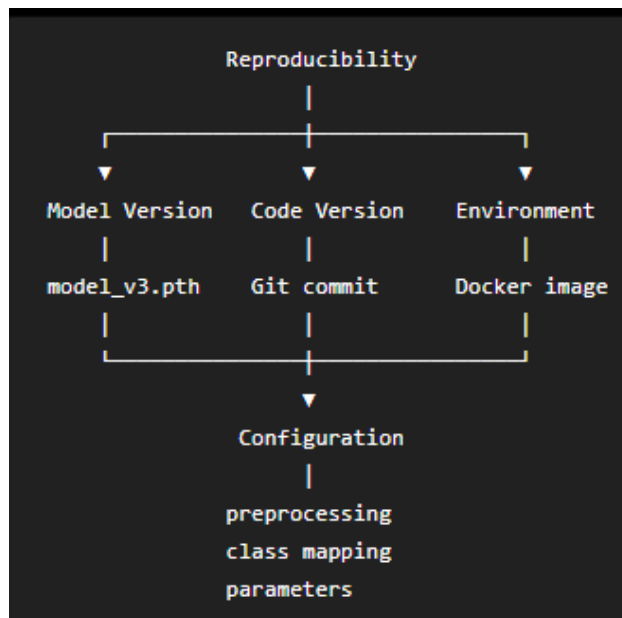
- Accuracy
- Precision
- Recall
- F1-Score
- Confusion matrix
- Per-class performance
- Confidence distribution

Edge cases:

- Empty input
- Invalid shape
- Corrupted input
- Very large input
- Unknown class
- Missing fields

Reproducibility:

I would version **everything required to reproduce the inference environment.**



I would maintain:

```
model/
  model_v3.pth
  model_metadata.json

src/
  model.py
  preprocessing.py
  inference.py
  api.py

tests/
  test_inference.py
  golden_inputs/

requirements.txt
Dockerfile
config.yaml
```

Metadata could maintain:

```

</> JSON

{
  "model_name": "customer_classifier",
  "model_version": "3.0.0",
  "framework": "pytorch",
  "pytorch_version": "2.x",
  "input_size": [224, 224],
  "classes": ["cat", "dog", "horse"],
  "preprocessing_version": "1.2.0"
}

```

Dockerization:

Docker Image: Python, PyTorch, Application code, Dependencies, Model, Configuration.

Monitoring in production:

System metrics: CPU/GPU utilization, Memory, Request throughput, P50/P95/P99 latency, Error rate

ML metrics: Prediction distribution, Confidence distribution, Class distribution, Data drift, Model performance when ground truth becomes available.

Why do we use `model.eval()` during inference?

`model.eval()` is used in **PyTorch during inference** to switch the model from **training mode** → **evaluation mode**.

Some **neural-network layers behave differently during training and inference**.

Which layers are affected?

1. Dropout

During training:

Neurons

A	B	C	D	E
↓	↓	×	↓	×

Randomly deactivate neurons

Dropout intentionally turns off random neurons to reduce overfitting.

During inference, we want **all neurons active**:

A	B	C	D	E
↓	↓	↓	↓	↓

All active

`model.eval()` disables Dropout's random behavior.

2. Batch Normalization:

BatchNorm behaves differently:

Training:

- Uses statistics from the current mini-batch.
- Updates running mean/variance.

Inference:

- Uses the learned running mean/variance from training.

`model.eval()` tells BatchNorm to use those stored statistics.

Difference between `torch.no_grad()` and `torch.inference_mode()`?

```
torch.no_grad()
|
├─ No gradient computation
└─ Some autograd bookkeeping remains

torch.inference_mode()
|
├─ No gradient computation
├─ Less autograd bookkeeping
├─ No view tracking
└─ No version-counter updates
```

therefore, for a pure inference service, `inference_mode()` can be more efficient.

How would you reduce inference latency?

1. First measure the latency

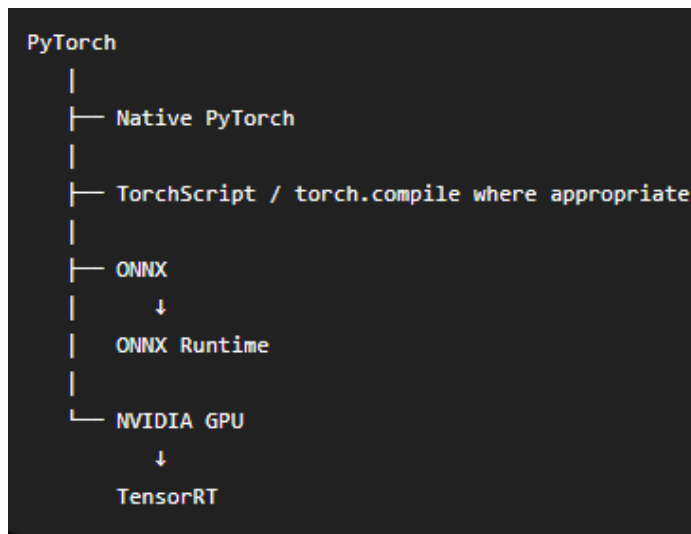
Measure: (average latency, p50, p95, p99, throughput, cpu/gpu utilization, memory usage)

For example:

```
Total latency = 100 ms

API      → 10 ms
Preprocess → 20 ms
Model    → 65 ms ← bottleneck
Postprocess → 5 ms
```

2. Use inference mode
3. Use GPU "cuda"
4. Use batching
5. Use mixed precision (FP16/BF16)
6. Optimize the model (FP32 - INT8)
7. Use optimized inference runtime



8. Optimize preprocessing (image decoding, Resizing, Tokenization, Serialization/Deserialization, CPU-GPU transfers, unnecessary data copies)
9. Optimize the API/service layer (connection pooling, Keep-alive connections, Efficient serialization, Request batching, Worker configuration, Avoiding unnecessary middleware, Async I/O where appropriate, Model warm-up)
10. Model serving architecture
11. Latency vs throughput:
 How long does one request take? Latency (Request -> prediction = 50ms)
 How many requests can the system process per second? 1000 requests / sec

Q4. Improving LLM Response Quality without fine-tuning

Sno	Technique	What it does	When to use	Key benefits
1	Prompt Engineering	Gives clear role, task, context, constraints, and instructions	Poor instruction following / inconsistent style	Better and more consistent
2	Few-shot prompting	Provides example of desired input - output behavior	Model needs to follow a specific format/style	Improves instruction
3	RAG	Retrieves relevant external knowledge and gives it to the	Domain specific or frequently changing	Reduce
4	Better Chunking	Splits docs into meaningful	Poor retrieval / context	Improves retrieved context
5	Better Embedding	Represent queries/docs semantically	Poor semantic search	Improves retrieved context
6	Hybrid	Combines keyword + semantic search	Exact terms and semantic meaning both matter	Better recall
7	Reranking	Reorders retrieved documents by relevance	Too many/irrelevant retrieved chunks	Better context
8	Context Management	Provides only relevant information to the LLM	Large/noisy context	Reduces confusion and token usage
9	Tool Calling	Allows LLM to access APIs/databases/tools	Real-time or transactional data	More accurate, current information
10	Structured Output	Forces predictable response	API/agent/data extraction use cases	Consistent machine readable output
11	Guardrails	Validates input/output and restricts unsafe behavior	Safety, compliance, hallucination	Safer responses

12	Temperature Turning	Controls response	Need deterministic vs creative output	Better consistency
13	Model Selection	Uses the right LLM for the	Current model isn't	Better quality/cost trade-off
14	Evaluation	Measure response quality systematically	Need objective quality measurement	Identifies
15	Human Feedback	Uses real user/agent	Production improvement	Finds real-world failure cases
16	Monitoring	Tracks quality and failure	Production systems	Continuous improvement

Q5: When would you choose RAG over fine-tuning?

Aspect	RAG	Fine-Tuning
Full form	Retrieval-Augmented Generation	Fine-Tuning
Changes model weights?	✗ No	✓ Yes
Adds external knowledge?	✓ Yes	⚠ Can, but not ideal for frequently changing knowledge
Good for current information?	✓ Excellent	✗ Poor choice
Updates knowledge easily?	✓ Update documents/index	✗ Usually retrain/fine-tune again
Domain-specific behavior	✓ Somewhat	✓ Excellent
Style/format adaptation	✓ Possible	✓ Excellent
Hallucination reduction	✓ Strong when well-grounded	✗ Not guaranteed
Traceability/citations	✓ Easy	✗ Difficult
Training required	✗ No	✓ Yes
Infrastructure complexity	Retrieval infrastructure	Training/fine-tuning infrastructure
Cost of updates	Low	Higher
Best use case	Knowledge grounding ↓	Behavior/task specialization

Interview Overview final

11 August 2026 15:28

Variable is a name used to store/reference a value in memory.

A **list** is an ordered and mutable collection that can contain duplicate elements.

Tuple is ordered and Immutable collection that can contain duplicate elements.

A **set** is a mutable collection of unique elements that does not support indexing like a list.

A **dictionary** is a mutable collection that stores data as key-value pairs.

A **function** is a reusable block of code that performs a specific task. It can accept input parameters and optionally return an output.

***args** allows a function to accept a variable number of positional arguments. Behave like tuple.

****kwargs** allows a function to accept a variable number of keyword arguments. Behave like dictionary

A **Decorator** is a function that modifies or extends the behavior of another function without changing the original function's source code.

Exception handling is a mechanism in python used to detect, handle, recover from runtime errors without abruptly terminating the program

Exception handling allows your program to handle unexpected situations gracefully instead of crashing.

An **iterator** is an object that allows you to traverse through a collection of one element at a time using the `__iter__()` and `__next__()` methods.

A **lambda function** is a small, anonymous function written in a single expression using the lambda keyword.

Map() applies a function to every element of an iterable and return iterator containing the transformed values

Filter() selects elements from an iterable based on whether a function return True or False.

Multithreading is a technique where multiple threads execute within the same process and share the same memory space.

Multiprocessing is a technique where multiple independent processes execute tasks, each with its own memory space and python interpreter.

Deep learning:

Bias: model is too simple - underfitting

Variance: model is too sensitive - overfitting

Overfitting: machine learning model learns the training data too well, including noise and unnecessary patterns.

Imagine a student who memorizes all the answers from previous exam papers but doesn't understand the concepts.

When the same questions appear:

✓ Excellent performance

When new questions appear:

✗ Poor performance

This is similar to **overfitting**.

Underfitting: When a model is too simple to learn the underlying patterns in the data.

The model performs poorly on both training and testing data.

Imagine a student who only studies a few basic concepts and doesn't understand enough of the syllabus.

When the exam comes:

✗ Poor performance on both familiar and new questions

This is similar to **underfitting**.

Activation function: introduce non-linearity allowing neural network to learn complex pattern.

Regularization is a technique that adds a penalty to the loss function to reduce overfitting and improve a model's ability to generalize.

Cross-validation is a model evaluation technique used to estimate how well a machine learning model will perform on **unseen data**.

Data Augmentation: Rotation, Scaling, Cropping, Flipping where physically valid, Brightness changes, Noise, Translation.

Explain asynchronous programming in FastAPI?

Without async

One request waits.

With async

100 users can wait on database/API calls simultaneously.

```
@app.get("/predict")
async def predict():
    result = await model.predict()
```

Benefits: Higher throughput, Lower latency, Better Scalability